# Fast but Wasteful: Power Analysis and Implications of Host Networking

JISU OK, KAIST, Republic of Korea
SUE MOON, KAIST, Republic of Korea

As datacenter networks evolve toward ultra-high-speed links, the energy footprint of host-side packet processing grows increasingly significant. While prior works have scrutinized the performance overhead of networking stacks, the interaction between energy efficiency mechanisms and networking remains underexplored. This paper presents a systematic study of the impact of CPU frequency scaling - both core and uncore - on host networking performance and power consumption. We analyze two dominant communication primitives in modern datacenters: the Linux TCP/IP stack and RoCEv2 RDMA. Our findings reveal that uncore components operate at excessively high frequencies, contributing to overall system power consumption substantially, and that recent Linux's core scaling governors make suboptimal decisions regarding target frequency. By bounding uncore frequency, we achieve up to 46.1% power savings with lower degree of throughput degradation (<11.6%) for TCP-based key-value store workloads. The significance of uncore also holds for RDMA: its offloaded design minimizes responder CPU activity, yet high bandwidth workloads still trigger uncore power draw.

CCS Concepts: • **Networks** → *Network performance evaluation*; *End nodes*; • **Hardware** → *Power and energy*.

Additional Key Words and Phrases: Host networking; power consumption; DVFS; frequency scaling; CPU uncore

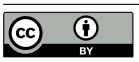
**ACM Reference Format:**
Jisu Ok and Sue Moon. 2026. Fast but Wasteful: Power Analysis and Implications of Host Networking. *Proc. ACM Netw.* 4, CoNEXT1, Article 4 (March 2026), 15 pages. https://doi.org/10.1145/3786287

## 1 Introduction

The ever-growing energy consumption of datacenter components is an ongoing, pressing challenge in both operational and environmental perspectives [10, 30]; networking is no exception. As access link bandwidth grows to hundreds of gigabits-per-second and beyond, the CPU consumption for host-side networking has kept up as well. Previous work has shown that more than one core is required to saturate a single 100 Gbps link [6], and such demands for CPU resources will only intensify with upcoming higher bandwidth technologies [1].

Energy-performance tradeoff in networking systems has been explored in the literature. On the one hand, DVFS has been the most viable option to enhance the efficiency of processor itself. However, a study over hundreds of efficiency-related configurations by Prekas *et al.* reassures that a system cannot achieve its full throughput at low operating frequency [44]. On the other hand, attempts to incorporate a new type of offloadable, energy-efficient device, such as SmartNIC, are emerging. While promising, Huang *et al.* dissect the regimes of its advantages: SmartNICs deliver better energy efficiency only at low packet rates [22]. Although the two studies are dissimilar in the targeted system components, there is a common implication: to satisfy SLO under the high

Authors' Contact Information: Jisu Ok, KAIST, Daejeon, Republic of Korea, jisu.ok@kaist.ac.kr; Sue Moon, KAIST, Daejeon, Republic of Korea, sbmoon@kaist.ac.kr.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2834-5509/2026/3-ART4
https://doi.org/10.1145/3786287

load, running a powerful enough processor at high frequencies is a must. This led us to question whether the current practical system is, first, raising the frequency *sufficiently* to handle the high load efficiently, and second, scaling its capacity *only as much as it needs*.

In this paper, we study the implications of CPU frequency scaling on end-host networking through the measurement of performance and power consumption in network-intensive workloads. In particular, along with core frequency scaling, we shed light on the significance of CPU uncore frequency, the less explored aspects in energy efficient networking. We examine the Linux kernel TCP/IP stack and RoCEv2 RDMA, the two representative communication primitives widely used in modern datacenter servers (we elaborate more on the rationale behind the exploration of RDMA in §4). We summarize our key findings as follows:

- **Network-intensive workload strongly stimulates uncore, resulting in excessively high uncore frequency.** Without manual intervention, the uncore frequency spikes up to near-maximum, significantly contributing to the system's power consumption. Through the evaluation with a key-value store application, we observe 43.3~46.1% power savings with 10.9~11.6% sacrifice on maximum throughput by throttling the frequency to 64% of the available maximum. Except for the highest system load, comparable degree of service latency is achievable with even lower frequency state.
- **The aggressiveness of the `ondemand` governor is effective in network processing.** The other two Linux scaling governors, `schedutil` and `conservative`, do not scale up core frequencies enough, delivering suboptimal efficiency.
- **Use of RDMA moves the protocol processing entirely to NIC, but power draw remains at CPU.** Though RDMA is a hardware-offloaded protocol stack, it still interacts with CPU uncore and how intensively it stimulates uncore affects CPU power consumption. In particular, bandwidth-intensive RDMA workloads raise the uncore frequency more than message-intensive ones.

## 2 Background

### 2.1 CPU Power Management in Linux

**Core frequency scaling.** Modern processors support a range of operating frequencies via DVFS (Dynamic Voltage and Frequency Scaling), and operating systems use DVFS to seek a balance between energy and performance. A high core frequency provides better performance but also consumes more power. In Linux, the `CPUFreq` subsystem is in charge of finding and configuring the efficient operating point of CPU cores. It has two key elements of scaling: governors and drivers. The scaling governors decide the target operating point (or P-state according to ACPI terminology [25]) based on core load and utilization data. Linux currently supports six generic scaling governors [50]: `performance`, `powersave`, `userspace`, `schedutil`, `ondemand`, and `conservative`. Among these, the last three dynamically estimate the core demand and calculate the desired frequency through their own mechanisms. In this work we use these three governors for evaluation. The scaling drivers interact with the processor hardware. They perform the actual frequency setting based on the decision made by the governors.

**Core idle state management.** When cores are not utilized, they enter idle states to save power rather than lowering the frequency (this is often referred to as "sleep"). In Linux, the `CPUIdle` subsystem manages the idle states. Multiple levels of idle states (or C-states according to ACPI terminology) exist, each of which has a different degree of turned-off components (C0 is an active state, and the idle state index starts from C1. The same multi-level idle states also exist for uncore).

In general, the deeper the idle state it is in, the more power it saves but also the longer it takes to wake up.

### 2.2 Uncore Power Management

**CPU uncore.** CPU consists of core and uncore[1], where the uncore part includes last-level cache (LLC), memory controllers (MCs) and on-chip interconnect (*e.g.*, Intel UPI). Uncore is shared by multiple cores and is regarded as a single entity per CPU.

**Uncore frequency scaling.** Similarly to core, uncore of recent processors also supports a frequency scaling feature that enables independent control of uncore performance and power [20, 46]. The difference is that the uncore frequency scaling is overseen by hardware and firmware[2], and its internals or algorithms on how it makes scaling decision is not disclosed. Several processor products, including the ones in our testbed, allow users to specify the range of frequencies on which the uncore scaling mechanism operates. If we set the lower and upper bounds to be the same, we effectively fix the frequency to a constant; our study uses this method to control the uncore frequency.

### 2.3 Remote Direct Memory Access

**RDMA.** To relieve the burden of CPU from heavy processing of host networking, there have been efforts to offload a part of or an entire networking stack. Remote Direct Memory Access (RDMA) is one such attempt. Thanks to its improved efficiency, RDMA has been widely deployed in modern datacenters [3, 17, 18] and supported by commodity NICs [14, 41]. We refer to RDMA-capable NICs as RNICs.

**RDMA semantics and Verbs.** Unlike the bytestream semantics of TCP, RDMA is a message-based protocol. It supports two types of distinct semantics, RDMA WRITE and READ, also known as RDMA Verbs.[3] An RDMA user either WRITEs a message to or READs a message from the memory at the remote side. An RNIC that initiates the RDMA Verbs is called a requester. An RNIC on the remote side of a connection that processes the requested Verb is called a responder. RDMA Verbs bypass kernel and are directly processed by RNIC. It does not require intervention of CPU at the responder.

**Datacenter networking primitives.** This paper addresses the two common networking primitives in today's datacenter, TCP and RDMA. TCP is still the de-facto standard protocol in Ethernet network. In the meantime, RDMA is broadening its popularity, especially in large-scale cloud providers. In this regard, we advocate a broad view on host networking: TCP/IP processing as well as HW-based RDMA protocol are both regarded as the host networking in this paper.

### 2.4 Measurement Methodology

**Metrics.** In this work the two main metrics to collect are throughput and latency and we collect them from workload generation tools. For configuration and power metrics, we collect CPU frequencies, utilization, and power consumption. Intel PCM [26] is used to obtain CPU utilization and frequency. System's power consumption is measured with `ipmitool` [35]. We use the same sampling rate (reading power every 1s) as in recent works [21, 22]. We report active power, defined by wall power (obtained from `ipmitool`) minus idle power, to account only for the amount consumed by the

[1]Throughout this paper, we use the term "core" and "uncore" as proper nouns to refer to core and uncore parts of CPU, respectively.

[2]Hardware-managed *core* frequency scaling is also available in recent platforms [51], but in this paper we limit our scope to the software-controlled core frequency scaling subsystem.

[3]There are other types of Verbs (*e.g.*, two-sided SEND and RECV) as well, but in this paper we focus only on WRITE and READ, the one-sided RDMA Verbs.

| H/W components | | | | S/W components | | | |
|---|---|---|---|---|---|---|---|
| CPU | 2x Intel Xeon Gold 5520+ | RAM | 256GB | OS | Ubuntu 24.04 | iperf3 | 3.17.1 |
| Core frequency | 0.8 ~ 2.0 GHz | NIC | NVIDIA ConnectX-6 Dx 100GbE | Driver | OFED 24.10-2.1.8.0 | perftest | 6.20 |
| Uncore frequency | 0.8 ~ 2.5 GHz | | | ipmitool | 1.8.19 | Valkey | 9.0.0 |
| | | | | PCM | 202405 Rel. | | |

Table 1. Testbed specification.

target workload and to rule out the impact of the surrounding environment (*e.g.*, temperature). Idle power is the power draw of a system with the lowest possible frequency (both core and uncore) and without any workload[4]. To further breakdown the system-wide power, we also measure CPU's and DRAM's power using PCM.

In this work we do not report the NIC power separate from the overall system power. NVIDIA ConnectX-6 Dx (NIC used in our testbed) does not offer a mechanism to report power consumption and its specification only states a power range of 19.52 to 26.64W [41]. Huang *et al.* have reported even NVIDIA BlueField SmartNICs use less than 8W of active power [22]. Thus we conclude that the active power of ConnectX-6 Dx NICs should also be less than 8W and has limited impact on the system-wide consumption.

**Testbed setup.** Table 1 shows the hardware and software components used in our experiments. We note that Intel Xeon Gold 5520+ supports the default operation range of 0.8 to 2.2 Ghz for core. However, at the base core frequency of 2.2 GHz, we have observed uncore frequencies spiking at random intervals. At 2.0 GHz, we have seen no such behavior and have chosen to bound the core frequency only up to 2.0 GHz.

Two servers with identical specifications in the table are directly connected with a 100 Gbps link without intervening switches. As in recent works [5, 21, 22], Turbo Boost [27] and Hyperthreading are disabled in our measurements to ensure deterministic and reproducible results.

## 3 Impact of Frequency Scaling on TCP networking

We begin our measurement study with the Linux TCP stack. We first demonstrate the significance of CPU *uncore* in power consumption and the impact of networking on uncore (§3.1). Then we study how the system behaves when controlled by different *core* scaling governors (§3.2). Lastly, we conduct a real-world application study and confirm *uncore*'s impact on the overall system power and performance. Our real-world application is a popular datacenter service, key-value store (§3.3).

### 3.1 Excessively High Uncore Frequency

**Uncore never sleeps.** The core frequency affects power consumption only as much as cores are utilized. Even if its frequency is set high, it consumes little power when it enters idle state and is not utilized. On the contrary, we observe that the uncore frequency contributes to power steadily and unwaveringly. Uncore does have the idle power management feature as well [8], but in none of our experiments it goes idle (*i.e.*, non-C0 states). When we measure the power of an idling machine from the `sleep` command, uncore still shows 100% active (*i.e.*, C0) state residency. This phenomenon has an important implication: if the uncore is configured to a high frequency, it will steadily consume high power regardless of the workload's uncore utilization.

[4]Although we measure idle power under Linux's default configuration (`schedutil` for the core scaling governor), which allows a range of core and uncore frequency in theory, we observe both core and uncore stay in the lowest-frequency state since there is no demand for scaling up.

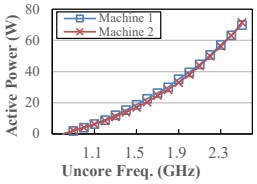
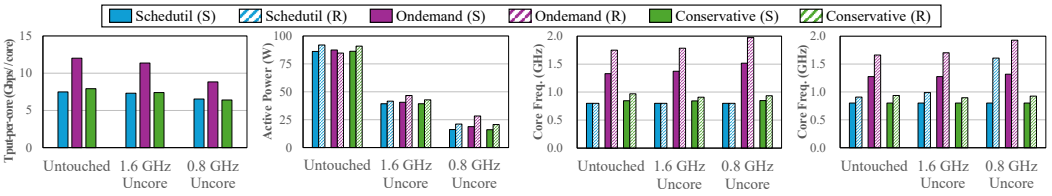



Fig. 1. Uncore power

| Governor | CPU | | | |
|---|---|---|---|---|
| | Sender | | Receiver | |
| | NIC-local | NIC-remote | NIC-local | NIC-remote |
| schedutil | 2.49 | 2.39 | 2.43 | 2.33 |
| ondemand | 2.49 | 2.40 | 2.21 | 2.09 |
| conservative | 2.49 | 2.39 | 2.34 | 2.24 |

Table 2. Uncore frequency (in GHz) induced by 12 TCP flows.




(a) Throughput-per-core (b) Power consumption (c) IRQ core frequency (d) `iperf3` core frequency

Fig. 2. System metrics over different *uncore* settings. `Untouched` represents system's default configuration, which allows uncore to run in a range of 0.8~2.5 GHz. (S: sender, R: receiver)

In Figure 1 we plot the system power consumption without any workloads, varying only the uncore frequency from 0.8 GHz to 2.5 GHz at 0.1 GHz intervals. Here the two machines are not engaged in networking. Because there is no direct method to isolate only uncore power from the overall system power, we use this result to estimate the uncore's contribution to total CPU power consumption in the later analysis (Figure 3).

**Workload.** Now we run networking workloads between the two machines and measure their impact on power consumption. We use `iperf3` [28] and focus on network processing; we minimize the effect of other application-specific computations [6]. With individual `iperf3` TCP flows pinned per core, along with their associated IRQ threads pinned to separate cores, we create 12 flows in total (using 24 cores) and run them for 60 seconds. While our CPU has 28 cores, we avoid using all of them at once to prevent system jitter. In our preliminary experiments we have observed non-deterministic performance when we used all 28 cores for flow processing. To ensure NUMA-affinity, only the NIC-local CPU is used. As the bottleneck is on the link, cores are not fully utilized. Therefore, we report aggregate throughput divided by total core utilization as a performance metric as in recent study [6].

**Uncore frequency peaks and plateaus.** Table 2 shows uncore frequencies for each CPU in our two-socket servers, measured with three core scaling governors. At the sender, the uncore of NIC-local CPU runs at 2.49 GHz in all cases, close to the available maximum (2.5 GHz). The receiver's NIC-local uncore runs at a slightly lower frequency, from 2.21 GHz to 2.43 GHz, depending on the governors. Interestingly, the uncore frequencies of the unused NIC-remote CPUs are also raised both at the sender and the receiver, reaching higher than 2 GHz. This interlocked frequency rise is consistent with the observation by Guo *et al.* [19].

The high uncore frequency, regardless of whether it was utilized (in NIC-local CPU) or not (in NIC-remote CPU), contributes heavily to the overall system power. Figure 2b (`Untouched`) shows the system-wide power consumption and Figure 3 shows its breakdown. The shaded area in Figure 3 represents the uncore's proportion within each CPU. We note that this uncore power is a projected value from the measurement in Figure 1. Uncore accounts for 74% of total power draw at the sender, 40% coming from the local CPU and 34% coming from the remote one. Its significance also holds for the receiver (61% from uncore).

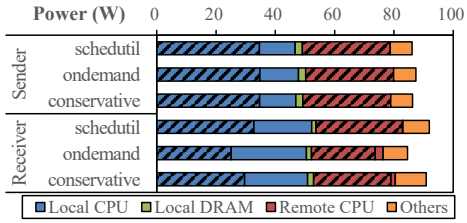



Fig. 3. Power breakdown of Untouched

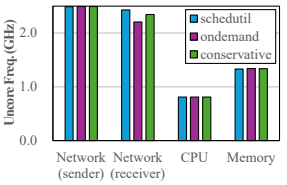



Fig. 4. Uncore frequency with different types of workload under Untouched setting.

**Is near-maximum uncore frequency necessary?** After observing the high power draw of uncore, we ask whether this near-maximum frequency is necessary. We repeat the experiment, while manually fixing the uncore frequencies, and present the various system metrics in Figure 2. Here, we focus on the impact of uncore; the differences coming from the selected governors are discussed in §3.2.

For all three governors, when uncore operates at 1.6 GHz, throughput-per-core remains comparable (Figure 2a, 2.3~6.6% decrease) but with notable power saving (Figure 2b, 44.8~54.8% reduction). If we further lower the uncore frequency to 0.8 GHz, we see 12.6~26.5% lower performance but with 66.6~81.6% reduced power. These results imply that under the default setting, uncore was operating at an excessively high frequency, aggravating the energy efficiency.

**Networking stimulates uncore more than other types of workload.** Figure 4 compares the measured uncore frequency of network-intensive (the `iperf3` workload), CPU-intensive, and memory-intensive workloads. For the latter two, `stress-ng` tool is used, with parameters of 100% load (CPU-intensive) or 256MB memory (memory-intensive) per worker [34]. We launched 24 workers in both workloads to match the number of used cores with the network-intensive one. We let uncore operate in the default frequency range (0.8~2.5 GHz) and present results from the NIC-local socket only.

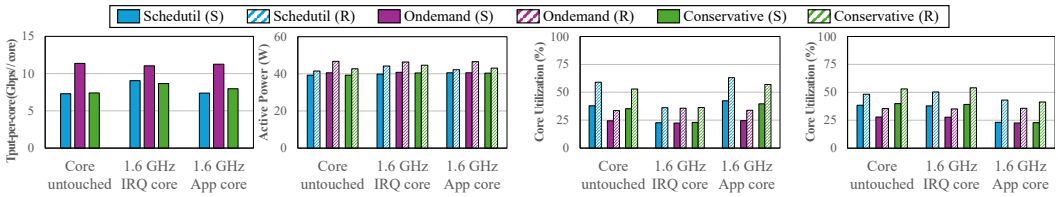
Network-intensive workload stimulates uncore most aggressively. The uncore frequency remains almost unchanged (0.81 GHz) in the CPU-intensive workload. We tested with different levels of CPU load (25%, 50%, and 75%), but results were the same. The memory-intensive workload raises the frequency to 1.33~1.34 GHz, but it is far lower than the near-maximum of the network-intensive one. We also tested different volumes of memory load per worker (1024MB and 4096MB) but they reached lower frequencies (0.98~1.1 GHz).

We surmise that Direct Cache Access (DCA) is one possible reason for this observation. Though the internals of uncore frequency control is a blackbox, a recent study has revealed that frequent LLC accesses result in higher uncore frequencies [19]. Given that DCA plays an important role in high-speed networking [6, 16, 48] and it is enabled in our setup, frequent read and write of LLC at the sender and the receiver, respectively, by the NIC may cause the uncore frequency being boosted.

### 3.2 Aggressive ondemand governor

Now we investigate the distinct effect of the three Linux scaling governors. All of them monitor CPU cores and adjust the core frequency: `schedutil` uses a moving average of CPU utilization in scheduler context, while `ondemand` and `conservative` use time-based CPU load estimation over a fixed window [50]. The difference between the latter two lies in how one adjusts the frequency: `ondemand` does it aggressively and `conservative` moves it gradually.

**Workload.** The same workload as in §3.1 is used.




(a) Throughput-per-core (b) Power consumption (c) IRQ core utilization (d) App core utilization

Fig. 5. System metrics over different *core* frequency setting. Uncore frequency is fixed to 1.6 GHz. In `Core untouched`, governor is allowed to adjust core frequency in a range of 0.8~2.0 GHz. (S: sender, R: Receiver)

**Aggressiveness of `ondemand` is effective.** As the `ondemand` governor is the most aggressive among the three, Figure 2a shows that it delivers the highest throughput per core regardless of the uncore frequencies (*e.g.*, 60.6% better than `schedutil` at the default uncore setting). Yet, it is for the `ondemand` governor that the lower uncore frequencies have greater influence on the performance.

Figure 2c and Figure 2d show the average frequencies of IRQ and `iperf3` cores. Under `schedutil` and `conservative` governors the core frequencies remain almost constant, while `ondemand` adjusts to the high frequencies. We note here that with `ondemand` the core frequencies have gone up more than other governors as the uncore frequencies are lowered to 1.6 GHz and 0.8 GHz. All three governors consume more or less equal power as we see in Figure 2b. Knowing that `schedutil` is better integrated with the kernel scheduler and should better estimate CPU demand [4], we find its low throughput per core and low core frequencies rather surprising.

**Are `schedutil` and `conservative` scaling up enough?** To evaluate whether the two governors' scaling decisions are the optimum in terms of efficiency, we manually raise the core frequency (either of IRQ cores or application cores) and present results in Figure 5. The uncore frequency is fixed to 1.6 GHz[5] to rule out the effect of uncore variability. If we boost IRQ cores to 1.6 GHz with `schedutil`, receiver core utilization shows 20.8%p net reduction (22.8%p decrease in IRQ cores and 2.0%p increase in application cores), resulting in a 24.1% improvement of throughput-per-core. A similar improvement is observed for `conservative` (17.1%). In both cases, the increase in power consumption is limited (less than 2.7W). Boosting application cores also shows difference (1.1% and 7.7% better throughput-per-core for `schedutil` and `conservative`, respectively), but its significance is less than IRQ core boosting. These results imply that `schedutil` and `conservative` were not making optimal scaling decisions for IRQ cores.

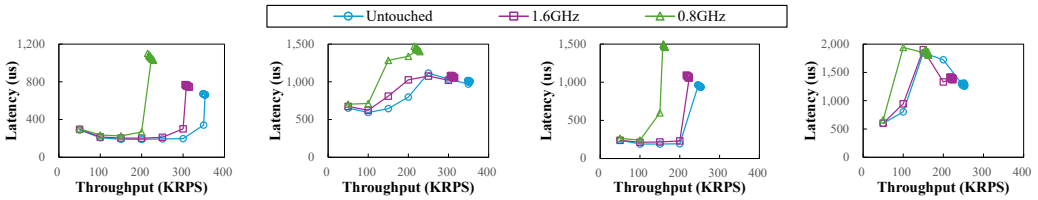
### 3.3 Application Case Study: Key-Value Store

To evaluate whether the above observations also hold for real-world networked applications, we study `Valkey`[6] key-value store (KVS) as a representative workload. We have chosen KVS for several reasons. First, KVS has been a popular and essential service in datacenters. A great diversity of prior works also studied KVS [2, 37, 44, 45, 52]. Second, KVS deployment is highly network-intensive, spending most of its execution time in network processing [37, 44]. Lastly, KVS exhibits a bursty pattern, which allows us to better understand the real-world networking behavior in datacenter [2, 7, 29].

**Workload.** We evaluate the performance and energy behavior of `Valkey` server under varying system load. YCSB is used for workload generation [13]. We first populate 10 million records to a

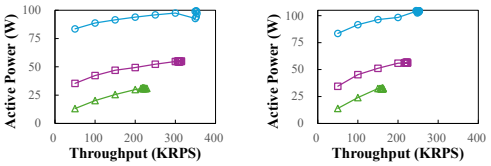
[5]We choose 1.6Ghz because it shows comparable performance and core frequencies to `untouched` in Figure 2.

[6]`Valkey` [12] is a high performance KVS, recently forked from Redis [40]. As a result, it shares many design components and characteristics with Redis including the protocol used between client and server. We chose Valkey because of its native support for multithreading (limited to I/O) and RDMA.



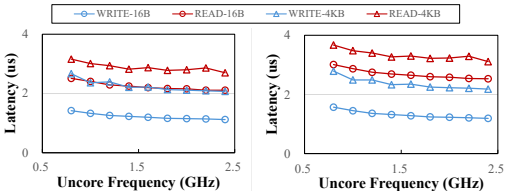

(a) 256B - P50 (b) 256B - P99 (c) 4KB - P50 (d) 4KB - P99

Fig. 6. Median (P50) and tail (P99) latency of Valkey read requests over varying system load. Request keys follow Zipfian distribution.




(a) 256B (b) 4KB

Fig. 8. Power consumption of Valkey workload.




(a) P50 (b) P99

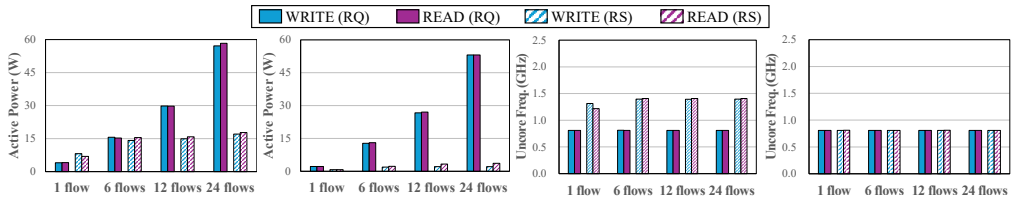
Fig. 9. RDMA message latency.

Valkey store. The dominant value size in KVS deployment differs by usage scenario; while several works report up to a few hundreds bytes of small values as a majority [2, 52], relatively large value sizes are also reported (*e.g.*, photo caching server [23]). Thus we experiment two types of record size: 256B and 4KB. Then we perform 50 million operations in total and measure the throughput and latency. 24 YCSB processes are launched, each with 10 threads. The workload consists of 99% read and 1% write operations considering that read mostly dominates datacenter key-value queries [2]. For key selection, we test two types of distribution: Zipfian and uniform. Unlike `iperf3` workload, the Valkey workload used here does not saturate the network link.

**Valkey also boosts uncore frequency.** Similar to §3.1, we first run the workload with uncore running in a default range of frequencies (Untouched). Core frequency is managed by ondemand since it was the most effective governor in §3.2. In all measurements with different record sizes, key distributions, and system load levels, the uncore frequencies of Valkey server were higher than 2.1GHz.

**Excessively high uncore frequency in Valkey.** Then we manually lower the uncore frequency to 1.6 GHz and 0.8 GHz, and plot throughput-latency curve in Figure 6. We only present for Zipfian here, but the overall implication is same for the uniform distribution; §A shows results for the uniform. Power consumption with each configuration is presented in Figure 8. Overall, low uncore frequencies degrade the maximum achievable throughput of a Valkey server but also reduce the power consumption notably. Fixing at 1.6 GHz saves 43.3~46.1% power only with 10.9~11.6% throughput loss, making it a justifiable trade-off for energy efficiency.

Even at low load, the unthrottled uncore peaks to the high frequency and consequently consumes high power. Such powerful uncore states are unnecessary because the comparable level of service latency is achieved with lower frequencies. For instance, for 256B requests, 0.8 GHz uncore provides comparable median and tail latency until 100 KRPS, but with 77.2~84.3% less power consumption. Beyond that point, the tail latency of 0.8 GHz uncore spikes up but 1.6 GHz uncore is now on a par with the unthrottled case with 45.5~48.7% less power. The advantage of unthrottled uncore on latency becomes clear starting from 300 KRPS. A similar tendency also holds for 4KB requests while the exact inflection points of load level differ. The implication here is that the uncore capacity




(a) Power of Bw-apps. (b) Power of Msg-apps. (c) Uncore of Bw-apps. (d) Uncore of Msg-apps.

Fig. 10. Power consumption and uncore frequency of Bw-apps and Msg-apps. (RQ: requester, RS: responder)

required to keep the latency objective depends on the system load and the current `untouched` configuration does not take this into account, resulting in inefficient uncore scaling decision.

## 4 Impact on RDMA

Now we study the interplay between CPU frequency scaling and another emerging high-performance networking technology, RDMA. We use RoCEv2, a widely deployed RDMA protocol in Ethernet-based datacenter networks [3, 17, 18].

**Motivation.** In addition to performance, RDMA has potential in terms of energy consumption. RDMA's protocol stack is processed in hardware ASIC embedded in RNIC, which has the advantage in efficiency over general-purpose CPU cores.

Despite its offloaded nature, however, the usage of RDMA in practice does not completely rule out CPU in the data path. It is still the CPU core that ultimately orchestrates end-to-end communication. In particular, a majority of RDMA applications take a polling approach to notice the completion of requests as early as possible [32]. Such polling tasks are performed by application cores, typically in a busy-waiting manner. Moreover, RDMA also benefits from Direct Cache Access (DCA), and this necessitates the cooperation of RNIC and CPU uncore. Therefore, a better understanding of its interaction with the host CPU will aid in fully exploiting the efficiency merit of RDMA networking.

### 4.1 Power consumption in RDMA

**Workload.** As in recent works [36, 53], we classify RDMA traffic into three types - delay-sensitive, message-intensive, and bandwidth-intensive flows - and generate the flows using `perftest` [42], a widely used RDMA Verbs benchmarking application. The number of flows differs by experiments, but we always dedicate one core per flow. As in §3, only NIC-local cores are used.

**Scaling governor loses its effect in RDMA.** We first run bandwidth-intensive flows (*Bw-apps*, single 1MB message at a time per flow) and message intensive flows (*Msg-apps*, 32-batch of 16B messages per flow) and measure utilization and frequency of cores. Though we omit the entire suite of data here due to limited space (core utilization is shown in §B), results are straightforward: at the requester-side, cores always run at the maximum utilization and the highest frequency. This is because the governors view the polling-based RDMA application merely as a highly CPU-intensive task.[7] Meanwhile, at the responder-side, core utilization is nearly zero, so there is no target on which the governors act. The three governors behave identically here, so in the remainder of this section we only present the results with `schedutil` (the default governor of Linux in our testbed).

**RDMA relieves responder CPU from processing, but a hidden power cost remains.** Figure 10 shows the power consumption and uncore frequency with different types of workloads. Requester

[7]We acknowledge that the CPU-intensiveness caused by busy-polling is especially acute in our experiment because we use the benchmark application (`perftest`). Nevertheless, in real-world polling-based RDMA applications, app-specific processing will occupy a part of the core capacity and RDMA polling will eventually fill up the rest, forming full utilization.

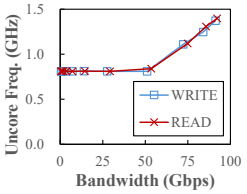



Fig. 11. Uncore freq.

| | Throughput (RPS) | P50 (ms) | P95 (ms) | P99 (ms) | Active power (W) |
|---|---|---|---|---|---|
| Untouched | 4,396 | 5.00 | 9.23 | 17.15 | 26.52 |
| 0.8GHz uncore | 3,351 | 6.92 | 10.56 | 17.17 | 7.42 |

Table 3. Performance and power consumption of RDMA-based `Valkey`.

power is dominated by core. At the requester-side, cores run at full capacity as explained above, but uncore frequency remains unchanged. As a result, the power consumption increases as the number of flows grows both for Bw-apps (Figure 10a) and Msg-apps (Figure 10b). On the other hand, how much high the uncore frequency is boosted dictates the responder power. Responder cores are relieved from heavy network processing, but RNIC still interact with CPU uncore. If that interaction raises the uncore frequency, it leads to extra power consumption. BW-apps in Figure 10a is the case, consuming 6.9~17.7W. However, Msg-apps do not induce uncore frequency boost, thus the power is low (less than 3.6W).

**Bandwidth intensity of responder RNIC stimulates uncore more than message intensity.** With 24 flows in Figure 10, the message processing capacity of RNIC is saturated with Msg-apps, reaching 54.4~56.6 Mmps in aggregate. Even under this highest message rate, responder uncore frequency remains unchanged. However, Bw-apps raised uncore frequency only with 0.01 Mmps. Except for the case of 1 flow, both WRITE and READ saturate the 100Gbps link.

To further understand the uncore stimulation by RDMA, we measure RNIC bandwidth and uncore frequency in Figure 11. We controlled the aggregate bandwidth by launching 24 Msg-apps and increasing the message size. Uncore frequency is affected by bandwidth only when it is beyond the certain threshold. In our testbed, the threshold is similar for WRITE and READ (51~53 Gbps).

### 4.2 Message latency and uncore frequency

Figure 9 shows message-level latency of RDMA over a range of uncore frequencies, for 16B and 4KB messages. Lowering the frequency increases latency at most 0.6 $\mu$s. For large messages (1MB), the measured latencies were an order of magnitude higher (92.62~93.06 $\mu$s), so the impact of the increased delay stemming from low uncore frequency is smaller.

### 4.3 RDMA-based Key-Value Store

Given that BW-apps raise the uncore frequency most, we seek the opportunity to reduce the power consumption of RDMA applications with minimal performance loss as in §3. To this end, we study the KVS again, with RDMA as underlying transport. RDMA-based KVS is a widely explored subject in both academia [31, 49] and practice [11].

**Workload.** The `valkey-benchmark` tool is used to generate `Valkey` requests from 24 client connections. We use SET workload to make the `Valkey` server RNIC running as a responder for the payload [11]. 1MB value size is used to evaluate bandwidth-intensive traffic.

**Throttling uncore frequency degrades throughput, but the impact on latency is limited.** We run the `Valkey` workload with the `untouched` and throttled uncore configurations and present results in Table 3. Latencies are reported values from `valkey-benchmark`. With `Untouched` setting, the measured uncore frequency was as high as 1.5 GHz. When we manually lower it to 0.8 GHz, the `Valkey` server's maximum throughput degrades 23.8% with 72.0% reduced power. While the higher median latency is observed, the tail latency remains comparable. This implies that at the

low load for RDMA-based `Valkey`, a similar level of tail latency objective is achievable with low uncore frequency and improved energy efficiency.

## 5 Related Work

**Networking stack performance and efficiency.** Prekas *et al.* explored the joint configurations of CPU efficiency features such as DVFS, Hyperthreading, and Turbo Boost and demonstrated superior performance of dynamic resource control for latency-sensitive applications [44]. A good body of work also addresses growing processing demands and efficiency of host network stacks and network-intensive workloads [5, 6, 21, 22, 24], but CPU frequency scaling is not taken into account. Performance was a major concern and CPU frequency was neither considered nor touched [6, 24], or core frequency was fixed to the maximum available value to maximize and stabilize host CPU performance [5, 21, 22]. While energy efficiency of networked systems was also explored in recent papers [21, 22, 39], their focus was to compare the efficiency of SmartNIC and host CPU at their best-performing configuration, and the interaction between dynamic frequency scaling and networking was not explored.

**CPU frequency scaling in other domains.** DVFS and core frequency scaling is a widely studied topic: server and datacenter systems [15, 43, 54], GPU systems [9, 47], and battery-powered devices [33, 38], just to name a few. A recent study by Schöne *et al.* analyzes energy-related features in a new generation of heterogeneous processor [46].

## 6 Conclusion

This paper has diagnosed the efficiency of current practices in networking systems and assessed whether there is any cost that we are unnecessarily or excessively paying from the energy perspective. Through extensive measurements on network-intensive workloads, we uncover that the CPU uncore subsystem remains overly aggressive, operating at high frequencies regardless of actual demand. This uncore behavior leads to significant power consumption, undermining system-wide efficiency. By bounding the uncore frequency, substantial power savings - up to 46.1% for KVS workloads - are achievable with minimal performance impact. For RDMA, although offloading reduces CPU usage, polling-based designs and bandwidth-induced uncore activity still incur notable energy costs. Our findings call for greater visibility and control over uncore behavior; energy-aware networking should consider not just protocol and stack design, but also architectural interactions across CPU subsystems.

## Acknowledgments

We thank our anonymous reviewers, Wonsup Yoon, and Seongkyu Lee for their valuable feedback. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2022-NR070293) and Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2025-02263378, Development of Heterogeneous AI Chip Virtualization Technology).

## References

[1] Ethernet Alliance. 2025. 2025 Ethernet Roadmap. https://ethernetalliance.org/wp-content/uploads/2025/03/2025-Ethernet-Roadmap-2-Sided-Web-03-17-2025.pdf.

[2] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. 2012. Workload analysis of a large-scale key-value store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems* (London, England, UK) *(SIGMETRICS '12)*. Association for Computing Machinery, New York, NY, USA, 53–64. doi:10.1145/2254756.2254766

[3] Wei Bai, Shanim Sainul Abdeen, Ankit Agrawal, Krishan Kumar Attre, Paramvir Bahl, Ameya Bhagat, Gowri Bhaskara, Tanya Brokhman, Lei Cao, Ahmad Cheema, Rebecca Chow, Jeff Cohen, Mahmoud Elhaddad, Vivek Ette, Igal Figlin,

Daniel Firestone, Mathew George, Ilya German, Lakhmeet Ghai, Eric Green, Albert Greenberg, Manish Gupta, Randy Haagens, Matthew Hendel, Ridwan Howlader, Neetha John, Julia Johnstone, Tom Jolly, Greg Kramer, David Kruse, Ankit Kumar, Erica Lan, Ivan Lee, Avi Levy, Marina Lipshteyn, Xin Liu, Chen Liu, Guohan Lu, Yuemin Lu, Xiakun Lu, Vadim Makhervaks, Ulad Malashanka, David A. Maltz, Ilias Marinos, Rohan Mehta, Sharda Murthi, Anup Namdhari, Aaron Ogus, Jitendra Padhye, Madhav Pandya, Douglas Phillips, Adrian Power, Suraj Puri, Shachar Raindel, Jordan Rhee, Anthony Russo, Maneesh Sah, Ali Sheriff, Chris Sparacino, Ashutosh Srivastava, Weixiang Sun, Nick Swanson, Fuhou Tian, Lukasz Tomczyk, Vamsi Vadlamuri, Alec Wolman, Ying Xie, Joyce Yom, Lihua Yuan, Yanzhao Zhang, and Brian Zill. 2023. Empowering Azure Storage with RDMA. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 49–67. https://www.usenix.org/conference/nsdi23/presentation/bai

[4] Neil Brown. [n. d.]. Improvements in CPU frequency management. https://lwn.net/Articles/682391/.

[5] Peter Cai and Martin Karsten. 2023. Kernel vs. User-Level Networking: Don't Throw Out the Stack with the Interrupts. *Proc. ACM Meas. Anal. Comput. Syst.* 7, 3, Article 49 (Dec. 2023), 25 pages. doi:10.1145/3626780

[6] Qizhe Cai, Shubham Chaudhary, Midhul Vuppalapati, Jaehyun Hwang, and Rachit Agarwal. 2021. Understanding host network stack overheads. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) *(SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 65–77. doi:10.1145/3452296.3472888

[7] Christopher Canel, Balasubramanian Madhavan, Srikanth Sundaresan, Neil Spring, Prashanth Kannan, Ying Zhang, Kevin Lin, and Srinivasan Seshan. 2024. Understanding Incast Bursts in Modern Datacenters. In *Proceedings of the 2024 ACM on Internet Measurement Conference* (Madrid, Spain) *(IMC '24)*. Association for Computing Machinery, New York, NY, USA, 674–680. doi:10.1145/3646547.3689028

[8] Paizhuo Chen, Lei Li, and Zhice Yang. 2021. Cross-VM and Cross-Processor Covert Channels Exploiting Processor Idle Power Management. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 733–750. https://www.usenix.org/conference/usenixsecurity21/presentation/chen-paizhuo

[9] Sangjin Choi, Inhoe Koo, Jeongseob Ahn, Myeongjae Jeon, and Youngjin Kwon. 2023. EnvPipe: Performance-preserving DNN Training Framework for Saving Energy. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 851–864. https://www.usenix.org/conference/atc23/presentation/choi

[10] McKinsey & Company. 2024. The role of power in unlocking the European AI revolution. https://www.mckinsey.com/industries/electric-power-and-natural-gas/our-insights/the-role-of-power-in-unlocking-the-european-ai-revolution.

[11] Valkey contributors. [n. d.]. RDMA experimental support. https://valkey.io/topics/RDMA/.

[12] Valkey contributors. [n. d.]. Valkey. https://valkey.io/.

[13] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing* (Indianapolis, Indiana, USA) *(SoCC '10)*. Association for Computing Machinery, New York, NY, USA, 143–154. doi:10.1145/1807128.1807152

[14] Intel Corporation. [n. d.]. Intel® Ethernet E810 Network Adapters. https://www.intel.com/content/www/us/en/products/details/ethernet/800-network-adapters/e810-network-adapters/products.html.

[15] Qingyuan Deng, David Meisner, Abhishek Bhattacharjee, Thomas F. Wenisch, and Ricardo Bianchini. 2012. CoScale: Coordinating CPU and Memory System DVFS in Server Systems. In *2012 45th Annual IEEE/ACM International Symposium on Microarchitecture*. 143–154. doi:10.1109/MICRO.2012.22

[16] Alireza Farshin, Amir Roozbeh, Gerald Q. Maguire Jr., and Dejan Kostić. 2020. Reexamining Direct Cache Access to Optimize I/O Intensive Applications for Multi-hundred-gigabit Networks. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 673–689. https://www.usenix.org/conference/atc20/presentation/farshin

[17] Yixiao Gao, Qiang Li, Lingbo Tang, Yongqing Xi, Pengcheng Zhang, Wenwen Peng, Bo Li, Yaohui Wu, Shaozong Liu, Lei Yan, Fei Feng, Yan Zhuang, Fan Liu, Pan Liu, Xingkui Liu, Zhongjie Wu, Junping Wu, Zheng Cao, Chen Tian, Jinbo Wu, Jiaji Zhu, Haiyong Wang, Dennis Cai, and Jiesheng Wu. 2021. When Cloud Storage Meets RDMA. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 519–533. https://www.usenix.org/conference/nsdi21/presentation/gao

[18] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. 2016. RDMA over Commodity Ethernet at Scale. In *Proceedings of the 2016 ACM SIGCOMM Conference* (Florianopolis, Brazil) *(SIGCOMM '16)*. Association for Computing Machinery, New York, NY, USA, 202–215. doi:10.1145/2934872.2934908

[19] Yanan Guo, Dingyuan Cao, Xin Xin, Youtao Zhang, and Jun Yang. 2023. Uncore Encore: Covert Channels Exploiting Uncore Frequency Scaling. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 843–855. doi:10.1145/3613424.3614259

[20] Daniel Hackenberg, Robert Schöne, Thomas Ilsche, Daniel Molka, Joseph Schuchart, and Robin Geyer. 2015. An Energy Efficiency Feature Survey of the Intel Haswell Processor. In *Proceedings of the 2015 IEEE International Parallel and Distributed Processing Symposium Workshop (IPDPSW '15)*. IEEE Computer Society, USA, 896–904. doi:10.1109/IPDPSW.

2015.70

[21] Jinghan Huang, Jiaqi Lou, Yan Sun, Tianchen Wang, Eun Kyung Lee, and Nam Sung Kim. 2023. Making Sense of Using a SmartNIC to Reduce Datacenter Tax from SLO and TCO Perspectives . In *2023 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE Computer Society, Los Alamitos, CA, USA, 28–42. doi:10.1109/IISWC59245.2023.00025

[22] Jinghan Huang, Jiaqi Lou, Srikar Vanavasam, Xinhao Kong, Houxiang Ji, Ipoom Jeong, Danyang Zhuo, Eun Kyung Lee, and Nam Sung Kim. 2024. HAL: Hardware-assisted Load Balancing for Energy-efficient SNIC-Host Cooperative Computing. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 613–627. doi:10.1109/ISCA59077.2024.00051

[23] Qi Huang, Ken Birman, Robbert van Renesse, Wyatt Lloyd, Sanjeev Kumar, and Harry C. Li. 2013. An analysis of Facebook photo caching. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* (Farminton, Pennsylvania) *(SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 167–181. doi:10.1145/2517349.2522722

[24] Jaehyun Hwang, Qizhe Cai, Ao Tang, and Rachit Agarwal. 2020. TCP ≈ RDMA: CPU-efficient Remote Storage Access with i10. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 127–140. https://www.usenix.org/conference/nsdi20/presentation/hwang

[25] UEFI Forum Inc. 2022. Advanced Configuration and Power Interface (ACPI) Specification. https://uefi.org/sites/default/files/resources/ACPI_Spec_6_5_Aug29.pdf.

[26] Intel. [n. d.]. Intel Performance Counter Monitor (Intel PCM). https://github.com/intel/pcm.

[27] Intel. [n. d.]. What Is Intel® Turbo Boost Technology? https://www.intel.com/content/www/us/en/gaming/resources/turbo-boost.html.

[28] iPerf3 authors. [n. d.]. iPerf3. https://iperf.fr/.

[29] Panpan Jin, Jian Guo, Yikai Xiao, Rong Shi, Yipei Niu, Fangming Liu, Chen Qian, and Yang Wang. 2019. PostMan: Rapidly Mitigating Bursty Traffic by Offloading Packet Processing. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 849–862. https://www.usenix.org/conference/atc19/presentation/jin

[30] Nicola Jones. 2018. How to stop data centres from gobbling up the world's electricity. *Nature* 561 (Sept. 2018), 163–166. Issue 7722. doi:10.1038/d41586-018-06610-y

[31] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2014. Using RDMA efficiently for key-value services. *SIGCOMM Comput. Commun. Rev.* 44, 4 (Aug. 2014), 295–306. doi:10.1145/2740070.2626299

[32] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. Design guidelines for high performance RDMA systems. In *Proceedings of the 2016 USENIX Conference on Usenix Annual Technical Conference* (Denver, CO, USA) *(USENIX ATC '16)*. USENIX Association, USA, 437–450.

[33] Seyeon Kim, Kyungmin Bin, Sangtae Ha, Kyunghan Lee, and Song Chong. 2021. zTT: learning-based DVFS with zero thermal throttling for mobile devices. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services* (Virtual Event, Wisconsin) *(MobiSys '21)*. Association for Computing Machinery, New York, NY, USA, 41–53. doi:10.1145/3458864.3468161

[34] Colin Ian King. [n. d.]. stress-ng (stress next generation). https://github.com/ColinIanKing/stress-ng.

[35] Duncan Laurie. [n. d.]. ipmitool. https://github.com/ipmitool/ipmitool.

[36] Sugi Lee, Mingyu Choi, Ikjun Yeom, and Younghoon Kim. 2024. PeRF: Preemption-enabled RDMA Framework. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 209–225. https://www.usenix.org/conference/atc24/presentation/lee

[37] Jacob Leverich and Christos Kozyrakis. 2014. Reconciling high server utilization and sub-millisecond quality-of-service. In *Proceedings of the Ninth European Conference on Computer Systems* (Amsterdam, The Netherlands) *(EuroSys '14)*. Association for Computing Machinery, New York, NY, USA, Article 4, 14 pages. doi:10.1145/2592798.2592821

[38] Chengdong Lin, Kun Wang, Zhenjiang Li, and Yu Pu. 2023. A Workload-Aware DVFS Robust to Concurrent Tasks for Mobile Devices. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking* (Madrid, Spain) *(ACM MobiCom '23)*. Association for Computing Machinery, New York, NY, USA, Article 19, 16 pages. doi:10.1145/3570361.3592524

[39] Ming Liu, Simon Peter, Arvind Krishnamurthy, and Phitchaya Mangpo Phothilimthana. 2019. E3: Energy-Efficient Microservices on SmartNIC-Accelerated Servers. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 363–378. https://www.usenix.org/conference/atc19/presentation/liu-ming

[40] Redis Ltd. [n. d.]. Redis. https://redis.io/.

[41] NVIDIA. [n. d.]. NVIDIA ConnectX-6 Dx Ethernet Adapter Cards User Manual. https://docs.nvidia.com/networking/display/connectx6dxen/specifications.

[42] OpenIB. [n. d.]. perftest. https://github.com/linux-rdma/perftest.

[43] Leonardo Piga, Iyswarya Narayanan, Aditya Sundarrajan, Matt Skach, Qingyuan Deng, Biswadip Maity, Manoj Chakkaravarthy, Alison Huang, Abhishek Dhanotia, and Parth Malani. 2024. Expanding Datacenter Capacity with DVFS Boosting: A safe and scalable deployment experience. In *Proceedings of the 29th ACM International Conference on*

*Architectural Support for Programming Languages and Operating Systems, Volume 1* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 150–165. doi:10.1145/3617232.3624853

[44] George Prekas, Mia Primorac, Adam Belay, Christos Kozyrakis, and Edouard Bugnion. 2015. Energy proportionality and workload consolidation for latency-critical applications. In *Proceedings of the Sixth ACM Symposium on Cloud Computing* (Kohala Coast, Hawaii) *(SoCC '15)*. Association for Computing Machinery, New York, NY, USA, 342–355. doi:10.1145/2806777.2806848

[45] Waleed Reda, Marco Canini, Lalith Suresh, Dejan Kostić, and Sean Braithwaite. 2017. Rein: Taming Tail Latency in Key-Value Stores via Multiget Scheduling. In *Proceedings of the Twelfth European Conference on Computer Systems* (Belgrade, Serbia) *(EuroSys '17)*. Association for Computing Machinery, New York, NY, USA, 95–110. doi:10.1145/3064176.3064209

[46] Robert Schöne, Markus Velten, Daniel Hackenberg, and Thomas Ilsche. 2024. Energy Efficiency Features of the Intel Alder Lake Architecture. In *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering* (London, United Kingdom) *(ICPE '24)*. Association for Computing Machinery, New York, NY, USA, 95–106. doi:10.1145/3629526.3645040

[47] Zhenheng Tang, Yuxin Wang, Qiang Wang, and Xiaowen Chu. 2019. The Impact of GPU DVFS on the Energy and Performance of Deep Learning: an Empirical Study. In *Proceedings of the Tenth ACM International Conference on Future Energy Systems* (Phoenix, AZ, USA) *(e-Energy '19)*. Association for Computing Machinery, New York, NY, USA, 315–325. doi:10.1145/3307772.3328315

[48] Minhu Wang, Mingwei Xu, and Jianping Wu. 2022. Understanding I/O Direct Cache Access Performance for End Host Networking. *Proc. ACM Meas. Anal. Comput. Syst.* 6, 1, Article 22 (Feb. 2022), 37 pages. doi:10.1145/3508042

[49] Xingda Wei, Rong Chen, and Haibo Chen. 2020. Fast RDMA-based Ordered Key-Value Store using Remote Learned Cache. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 117–135. https://www.usenix.org/conference/osdi20/presentation/wei

[50] Rafael J. Wysocki. 2017. CPU Performance Scaling. https://docs.kernel.org/admin-guide/pm/cpufreq.html.

[51] Rafael J. Wysocki. 2017. intel pstate CPU Performance Scaling Driver. https://docs.kernel.org/admin-guide/pm/intel_pstate.html.

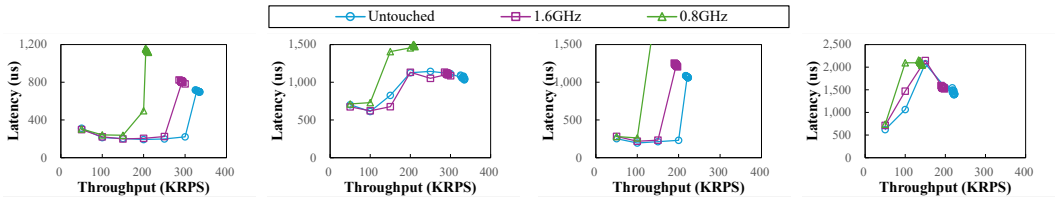
[52] Juncheng Yang, Yao Yue, and K. V. Rashmi. 2020. A large scale analysis of hundreds of in-memory cache clusters at Twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 191–208. https://www.usenix.org/conference/osdi20/presentation/yang

[53] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. 2022. Justitia: Software Multi-Tenancy in Hardware Kernel-Bypass Networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 1307–1326. https://www.usenix.org/conference/nsdi22/presentation/zhang-yiwen

[54] Liang Zhou, Laxmi N. Bhuyan, and K. K. Ramakrishnan. 2020. Gemini: Learning to Manage CPU Power for Latency-Critical Search Engines. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 637–349. doi:10.1109/MICRO50266.2020.00059

## A Throughput-latency of Valkey

Figure 12 presents the median and tail latency of Valkey server when the key distribution of requests is uniform.




(a) 256B - P50 (b) 256B - P99 (c) 4KB - P50 (d) 4KB - P99

Fig. 12. Median (P50) and tail (P99) latency of Valkey read requests over varying system load. Request keys follow uniform distribution.

## B Core utilization of RDMA flows

Table 4 shows core utilization of RDMA workloads. Regardless of flow numbers or traffic types, the requester utilization reaches the maximum while the responder remains unutilized.

| Flows | Client (Requester) | | | | Server (Responder) | | | |
|---|---|---|---|---|---|---|---|---|
| | Bw-WRITE | Bw-READ | Msg-WRITE | Msg-READ | Bw-WRITE | Bw-READ | Msg-WRITE | Msg-READ |
| 1 flow | 97.26 | 98.86 | 97.26 | 98.86 | 0.20 | 0.23 | 0.12 | 0.11 |
| 6 flows | 97.24 | 98.82 | 97.24 | 98.81 | 0.25 | 0.27 | 0.16 | 0.17 |
| 12 flows | 97.23 | 98.77 | 97.21 | 98.78 | 0.30 | 0.29 | 0.23 | 0.20 |
| 24 flows | 97.17 | 98.69 | 97.14 | 98.67 | 0.35 | 0.34 | 0.27 | 0.26 |

Table 4. Core utilization of RDMA flows.

Received June 2025; revised November 2025; accepted December 2025